

Mettre TriCV en service

Guide de déploiement — Kapi Consult · version du 14/09/2026

Ce document s'adresse à la personne qui installe et met TriCV en service. Il suppose un accès administrateur au serveur, et aucune connaissance du code de l'application. Chaque étape dit ce qu'il faut faire et, quand cela change la décision, pourquoi.

Deux documents l'accompagnent. Le **guide de configuration de la boîte de candidatures** détaille le paramétrage IMAP, qui est la partie la plus délicate ; il est cité au bon endroit. Le fichier **.env.example**, à la racine du dépôt, liste toutes les variables lues par l'application, chacune commentée.

1. Ce qu'il faut installer, et où

TriCV se découpe en deux moitiés qui n'ont ni le même public ni les mêmes exigences. Les séparer n'est pas une commodité technique : c'est ce qui permet d'exposer à Internet uniquement ce qui doit l'être.

Moitié	Qui l'utilise	Où la placer
Faces publiques page des avis, formulaire de candidature, espace de suivi du promoteur	Les candidats et les clients, depuis Internet	Un petit serveur hébergé, joignable publiquement, avec un nom de domaine et un certificat HTTPS
Outil interne grilles, dossiers, notes, entretiens, rapports	Les chargés de recrutement du cabinet	Le réseau du cabinet, ou le même serveur derrière une restriction d'accès

Les deux moitiés sont un seul programme et partagent une seule base de données : il n'y a qu'une installation à faire. Ce qui se sépare est **l'exposition** — quelles adresses sont ouvertes depuis Internet.

Ce qui ne doit jamais être exposé

Les adresses commençant par **/api/v1/** autres que **/api/v1/public/** et **/api/v1/espace-client/** donnent accès aux dossiers de tous les candidats. Si l'outil interne est mis en ligne, il doit l'être derrière une restriction — VPN, filtrage par adresse IP, ou authentification du serveur web en amont. Un mot de passe applicatif seul ne suffit pas à justifier d'ouvrir cette surface à Internet.

Prérequis

- **Docker et Docker Compose** — la voie recommandée : tout est fourni, y compris PostgreSQL et les modèles de reconnaissance d'entités.
- **Ou bien** Python 3.12, Node 20 et PostgreSQL 15, pour une installation sans Docker.
- Un **nom de domaine** pointant sur le serveur, et un certificat HTTPS. Les candidats déposent des pièces d'identité : le chiffrement du transport n'est pas optionnel.

Python 3.13 : à éviter

spaCy 3.8 ne le prend pas en charge. L'application fonctionne, mais l'expurgation des données personnelles retombe sur des expressions régulières et laisse passer certains noms de villes isolés. Utilisez Python 3.12, ou l'image Docker qui embarque la bonne version.

2. Installation

2.1 — Récupérer le dépôt et préparer la configuration

```
git clone <adresse-du-depot> tricv
cd tricv
cp .env.example .env
```

Le fichier **.env** n'est jamais versionné : il contient les secrets de cette installation. Le **.env.example** reste la référence de ce qui existe.

2.2 — Poser les secrets

Trois valeurs livrées avec le dépôt doivent changer avant toute mise en service. Le contrôle de l'étape 5 refuse de valider tant qu'elles sont en place.

Variable	Ce qu'elle protège	Comment la produire
JWT_SECRET	Les jetons de session. Qui le connaît peut se faire passer pour n'importe quel utilisateur et lire tous les dossiers.	<code>python -c "import secrets; print (secrets.token_urlsafe(48))"</code>
SEED_ADMIN_PASS WORD	Le premier compte administrateur.	Un mot de passe long, choisi pour cette installation, transmis à son titulaire hors du fichier.
POSTGRES_PASSWO RD et DATABASE_URL	L'accès direct à la base.	Un mot de passe propre, recopié à l'identique dans les deux variables.

Changer **JWT_SECRET** déconnecte tout le monde

C'est sans gravité, mais cela se produit une fois : faites-le avant d'ouvrir l'application à l'équipe, pas après.

2.3 — Adresses et exposition

- **URL_PUBLIQUE** — l'adresse par laquelle candidats et clients atteignent l'application, par exemple `https://recrutement.kapiconsult.tg`. Elle sert à construire les liens envoyés par courriel. Le serveur ne connaît que son adresse d'écoute et ne peut pas la deviner : sans cette variable, l'ouverture d'un accès client n'a pas de lien à envoyer.
- **CORS_ORIGINS** — la liste, séparée par des virgules, des adresses depuis lesquelles l'interface appelle l'API. Elle doit inclure l'adresse publique, et le site du cabinet si le formulaire de candidature y est intégré.
- **VITE_API_URL** — l'adresse de l'API, inscrite dans l'interface au moment de sa construction. La changer demande de reconstruire l'interface.

2.4 — Démarrer

```
docker compose up -d
docker compose exec api alembic upgrade head
```

La seconde commande crée le schéma de la base. Elle est **obligatoire** et se rejoue sans risque : elle n'applique que ce qui manque.

Sans Docker

Créez la base PostgreSQL, installez les dépendances (`pip install -r backend/requirements.txt`), puis lancez `alembic upgrade head` depuis `backend/`. L'interface se construit avec `npm ci && npm run build` depuis `frontend/` ; servez le contenu de `frontend/dist` avec le serveur web de votre choix.

3. Le courriel

TriCV lit une boîte pour y récupérer les candidatures, et en écrit une pour les convocations et les accès clients. Les deux se règlent depuis l'application — **Paramètres** — et non dans le fichier de configuration : l'adresse de recrutement change avec les campagnes, et un mot de passe d'application se révoque sans prévenir. Demander un accès au serveur à chaque fois condamnerait la fonction à ne pas servir.

3.1 — Réception (IMAP)

Le paramétrage complet, avec les pièges de Gmail et la façon de vérifier que le relevé fonctionne, fait l'objet d'un document séparé : **Guide-configuration-boite-candidatures.pdf**. Suivez-le avant de revenir ici.

3.2 — Envoi (SMTP)

Dans **Paramètres** > **Envoi de courriels**, renseignez le serveur, le compte et le mot de passe, puis l'adresse d'expédition — celle que les candidats verront. Le bouton **Tester l'envoi** ouvre une session sans rien expédier : une erreur d'identifiants doit se découvrir là, et non au moment où trente convocations partent.

Fournisseur	Serveur	Port	Mot de passe
Gmail / Google Workspace	smtp.gmail.com	587	Un mot de passe d'application , jamais celui du compte. Il se crée dans Compte Google > Sécurité , après avoir activé la validation en deux étapes.
Microsoft 365	smtp.office365.com	587	Le compte doit avoir l'authentification SMTP autorisée par l'administrateur du tenant ; elle est désactivée par défaut.
Hébergeur du domaine	Fourni par l'hébergeur	587 ou 465	Le mot de passe de la boîte. Le port 465 chiffre dès l'ouverture et ignore le réglage STARTTLS.

Sans configuration d'envoi, rien ne se perd

Les convocations se rédigent quand même dans l'application et se recopient à la main ; l'ouverture d'un accès client affiche le lien d'activation à transmettre soi-même. L'application **refuse** d'envoyer plutôt que de faire disparaître un message en silence.

4. Les comptes

Un compte donne accès aux dossiers de **tous** les candidats de **tous** les mandats. Il n'existe pas de cloisonnement par chargé de recrutement : c'est un choix assumé pour un cabinet de cette taille, mais il rend la création de comptes une décision, pas une formalité.

- Le premier compte administrateur est créé au démarrage à partir de **SEED_ADMIN_EMAIL** et **SEED_ADMIN_PASSWORD**.
- Les suivants se créent en ligne de commande, ou via l'inscription libre si elle est ouverte temporairement.
- L'inscription libre se referme dans **Paramètres** dès que l'équipe a ses comptes. Ouverte sans code, elle laisse quiconque atteindre la page lire tous les dossiers.

Gérer les comptes en ligne de commande

```
docker compose exec api python comptes.py lister
docker compose exec api python comptes.py promouvoir adresse@exemple.tg
docker compose exec api python comptes.py desactiver adresse@exemple.tg
```

L'outil refuse de retirer le dernier administrateur actif : sans cette garde, une erreur de manipulation fermerait définitivement l'accès aux réglages.

5. Le contrôle avant mise en service

Une installation qui démarre n'est pas une installation prête. Un serveur peut démarrer parfaitement avec le secret du dépôt, le mot de passe administrateur d'origine et onze candidats fictifs en base — et perdre les données du premier candidat réel qui se présente. Le contrôle répond à une seule question : **cette installation peut-elle recevoir de vrais dossiers ?**

```
docker compose exec api python preflight.py
```

Niveau	Ce que cela veut dire
BLOQUANT	À corriger avant de recevoir un dossier réel. Le contrôle sort en erreur tant qu'il en reste un.
À VOIR	Acceptable si c'est un choix ; à connaître sinon. Par exemple une clé Gemini gratuite, dont les conditions autorisent l'entraînement sur ce qui est envoyé.
OK	Vérifié.

Les données de démonstration sont bloquantes

Le dépôt installe onze candidats fictifs pour la prise en main. Mélangés à de vrais dossiers, ils faussent les grilles et le vivier. Le contrôle les détecte à leur adresse en @example.com et refuse de valider tant qu'ils sont là.

6. Ce qui sort de la machine, et ce qui n'en sort pas

Le classement des dossiers est entièrement calculé sur place : barème, éligibilité, notes, grilles. Aucun modèle de langage n'y intervient. L'assistance automatique sert à quatre choses seulement — lire un CV pour en proposer le parcours, juger l'équivalence d'un diplôme, rédiger un brouillon d'avis, proposer un texte de rapport — et ce qu'elle produit porte une provenance qui l'empêche d'éliminer quiconque sans relecture.

Réglage	Effet	Recommandation
PII_REDACTION	À <code>true</code> , le texte est extrait sur place et les identifiants — nom, adresse, téléphone, courriel — sont retirés avant tout envoi. À <code>false</code> , le fichier d'origine part tel quel.	<code>true</code> . Le passer à <code>false</code> est une décision à documenter.
REDACT_DEMOGRAPHICS	Retire l'âge, le sexe, la date de naissance et la nationalité du contenu envoyé pour la notation. Ces données restent visibles des chargés de recrutement et servent aux conditions d'âge ou de nationalité, qui sont évaluées sur place.	<code>true</code> . Noter sur ces attributs est discriminatoire dans la plupart des droits applicables.
LLM_PROVIDER	Le fournisseur qui lit les dossiers. <code>gemini</code> , <code>anthropic</code> , <code>ollama (local)</code> ou <code>openai</code> — ce dernier désignant une adresse, pas une marque : Mistral, Groq, OpenRouter, un serveur vLLM du cabinet.	Voir la section 6 bis. Toute offre gratuite s'entraîne sur ce qu'on lui envoie, sauf à le refuser quand c'est possible.
LLM_FALLBACK	Le fournisseur de secours, essayé quand le principal ne peut pas servir : allocation épuisée, ou service en panne après quatre tentatives. Dans ces deux cas seulement — une réponse illisible ou une clé refusée sont des défauts à voir, pas à contourner.	Utile si le cabinet dispose de deux clés. Lire la réserve de confidentialité en section 6 bis avant de l'activer.
LLM_PROSE_PROVIDER	Le fournisseur qui rédige les rapports, quand ce n'est pas celui qui dépouille.	Le plus sobre en invention. Un rapport part au client sous la signature du cabinet.
LLM_MAX_CONCURRENCY	Le nombre d'appels simultanés au fournisseur.	1 sur une offre gratuite limitée à une requête par seconde — au-delà, les appels partent en rafale, se font refuser, et attendent la reprise.

6 bis. Choisir le fournisseur du modèle

Le chiffre qui décide n'est pas le nombre de requêtes par jour. Un dépouillement envoie jusqu'à 24 000 caractères de CV, soit environ 7 000 jetons : une offre généreuse en requêtes mais plafonnée en **jetons par jour** s'épuise bien avant d'avoir consommé ses requêtes. La rédaction, elle, coûte quelques centaines de jetons par section.

Fournisseur	Offre gratuite	Dossiers par jour
Mistral	1 requête/s. La famille <code>ministral</code> seule est allouée : <code>mistral-small</code> et <code>medium</code> sont à zéro et répondent 429.	Plusieurs milliers
Gemini 2.5 Flash-Lite	15 req/min, 1 000 req/jour	~1 000

Fournisseur	Offre gratuite	Dossiers par jour
Gemini 2.5 Flash	10 req/min, 250 req/jour	~250
Groq llama-3.3-70b	30 req/min mais 100 000 jetons/jour	~13
Ollama, sur place	aucune limite	illimité

Mesurer avant de faire confiance

Un fournisseur qui coûte moins et lit moins bien n'est pas une économie : l'exactitude du dépouillement est ce sur quoi repose toute la présélection. Après chaque changement de fournisseur ou de modèle, lancer les deux contrôles ci-dessous. Le premier vérifie qu'une section de rapport revient en prose française et n'invente aucun nom ; le second mesure la lecture de six dossiers et se compare au repère inscrit dans le script (14/14 diplômes, 18/18 expériences).

```
cd backend
.venv/Scripts/python.exe tools/verifier_prose.py
.venv/Scripts/python.exe -m tools.evaluer_extraction essai
```

La confidentialité n'est pas la même partout. L'offre gratuite de Google s'entraîne sur ce qui lui est envoyé, sans réglage pour le refuser ; celle de Mistral aussi, mais le refus se pose dans la console (Admin > Privacy) sans changer d'offre. C'est à faire avant le premier dossier réel. Un secours qui descend de l'un vers l'autre déplace donc de vrais parcours d'un fournisseur qui les oublie vers un fournisseur qui les conserve : c'est une décision à prendre une fois, en connaissance de cause. Dans tous les cas, l'expurgation retire le nom, l'adresse, le téléphone, le courriel et la date de naissance avant l'envoi.

Deux clés valent mieux qu'une, et pas seulement pour le quota. Un fournisseur tombe en panne comme il s'épuise : un « 503, forte demande » qui survit aux quatre tentatives laisse l'application sans modèle, alors qu'une seconde clé en état de marche ne servirait à rien. Le secours prend le relais dans les deux cas, et le fournisseur écarté est mis de côté un quart d'heure plutôt que réinterrogé à chaque dossier. Le journal dit lequel des deux motifs a joué.

Ce qu'une bascule change, et qu'il faut savoir

Deux modèles ne lisent pas un dossier de la même façon. Un mandat dépouillé moitié par l'un, moitié par l'autre produit une grille dont les lignes ne viennent pas du même lecteur, et un candidat qui conteste son élimination a le droit de savoir lequel a lu son dossier. Chaque bascule est donc journalisée en nommant les deux fournisseurs.

Le modèle sur place. LLM_PROVIDER=ollama est la seule configuration où le dossier d'un candidat ne sort pas du bâtiment : ni clé, ni quota, ni conditions à relire. Le coût est en matériel et en temps de réponse. Un modèle de 7 à 8 milliards de paramètres en 4 bits occupe environ 5 Go et demande autant de mémoire vive graphique pour répondre en quelques secondes ; sans carte graphique il tourne sur le processeur, ce qui reste utilisable pour un traitement de nuit.

```
docker compose --profile local-llm up -d ollama
docker compose exec ollama ollama pull qwen3:8b

# puis dans .env :
LLM_PROVIDER=ollama
OLLAMA_MODEL=qwen3:8b
OLLAMA_BASE_URL=http://ollama:11434
```

Pour une carte NVIDIA, décommenter le bloc `deploy` du service `ollama` dans `docker-compose.yml` — il exige le NVIDIA Container Toolkit. Le laisser actif sans carte empêche le service de démarrer, d'où sa mise en commentaire par défaut.

7. Sauvegardes et exploitation

Deux choses sont à sauvegarder, et perdre l'une sans l'autre laisse un système incohérent : la base de données, et les fichiers déposés.

- **La base** — dossiers, notes, grilles, échanges, rapports. Sauvegarde quotidienne recommandée.
- **Les pièces** — le répertoire désigné par `STORAGE_PATH`, ou le compartiment S3 si `STORAGE_BACKEND=s3`.

```
# Base
docker compose exec -T db pg_dump -U tricv tricv > sauvegarde-$(date +%F).sql

# Pièces (stockage local)
tar czf pieces-$(date +%F).tar.gz /chemin/vers/STORAGE_PATH
```

Une sauvegarde jamais restaurée n'est pas une sauvegarde

Restaurer-la une fois sur une installation d'essai, avant d'en avoir besoin. C'est le seul moyen de savoir qu'elle est complète.

Maîtriser l'espace disque

Les pièces s'accumulent. Il n'existe aucune limite de taille au dépôt : refuser un dossier valable parce qu'un scan de diplômes est volumineux n'a pas de sens. L'espace se maîtrise autrement — en **purgeant les fichiers des mandats archivés** depuis **Archives > Purger les fichiers**. Les dossiers, les notes et les profils restent ; seuls les fichiers disparaissent, et le vivier garde de quoi retrouver une personne.

Mises à jour

```
git pull
docker compose build
docker compose up -d
docker compose exec api alembic upgrade head
```

La dernière commande est celle qu'on oublie. Sans elle, le code attend des colonnes que la base n'a pas, et les écritures échouent sans rien expliquer. Sauvegardez avant.

8. La première mise en service

Dans l'ordre, une fois l'installation faite :

- Effacer les données de démonstration et repartir d'une base vide.
- Relancer **preflight.py** jusqu'à n'avoir plus aucun point bloquant.
- Créer les comptes de l'équipe, puis refermer l'inscription libre.
- Configurer la boîte de candidatures, et faire un **aperçu** du relevé — il montre ce qu'un relevé ferait, sans rien créer.

- Configurer l'envoi, et le tester.
- Saisir un premier mandat réel, un poste, un avis — et vérifier que le lien de candidature s'ouvre depuis un appareil extérieur au réseau du cabinet.

Le point de non-retour

Dès qu'un candidat réel a déposé un dossier, l'installation n'est plus réinitialisable sans perte. Tout ce qui précède doit être fait avant de diffuser le premier lien de candidature.

TriCV classe et recommande ; les chargés de recrutement de Kapi Consult décident. Aucun dossier n'est écarté sur une donnée qu'une personne n'a pas confirmée.